

קורס 77315 - מבוא לפיסיקה חישובית (Computational Physics).

הקורס כולל סקירה של הבסיס המתמטי, אלגוריתמים ויישומים פיזיקליים מרכזיים ומטרתו הקניית ידע בסיסי בפיסיקה חישובית כולל התנסות אישית שתאפשר לסטודנט:

- לבחון בעיה פיזיקאלית ומידת התאמתה לפתרון ממוחשב.
- ליישם אלגוריתמים שונים לפתרון בעיות פיזיקאליות.
- לשקול מאפיינים שונים של פתרון נומרי כגון יציבות דיוק ויעילות.
- לבדוק נכונות של פתרון מחושב.
- לבחון היתכנות של חישוב מקבילי לפתרון בעיה פיזיקאלית ואפשרויות שונות למימוש.

הנושאים בקורס:

- שגיאות עיגול, דיוק ויציבות
- גזירה נומרית
- אינטרפולציה – לגרנז', הרמיט, cubic spline
- אינטגרציה נומרית – סימפסון, גאוס-לגנדר, שיטת רקורסיה
- שורש של פונקציה בממד אחד – "ציד אריות", ניוטון-רפסון, secant
- משוואות ליניאריות – אלימינציה של גאוס, מטריצות פס
- ריבועים מינימליים – אורתוגונליזציה של גרהם-שמידט
- מציאת ערכים עצמיים – שיטת Jacobi
- שורש של פונקציה רב ממדית – שיטת ניוטון-רפסון עם בקרה
- מציאת מינימום – שיטת השלשות בממד אחד, ירידה במורד ברב ממד
- משוואות דפרנציאליות רגילות – שיטות רב צעד, שיטות סתומות, predictor-corrector, רונגה-קוטה.
- בקרת שגיאה מצטברת, יציבות של פתרון, שיטת נומרוב
- בעיות תנאי שפה – shooting method, שיטת רלקסציה
- משוואות דיפרנציאליות חלקיות – הקדמה, בעיות תנאי שפה: Jacobi, Gauss-Seidel, multigrid
- בעיות תנאי התחלה – מרכז ויציבות של משוואות הפרשים
- משוואת דיפוזיה – פתרון מערכת טרידיאגונליות לממד אחד, פתרון מערכת רב ממדית
- משוואת אדבקציה – תנאי courant, leapfrog
- הידרודינמיקה – הקדמה, פתרון חד ממדי לגרנז', פתרון אויילרי, שיטת קרקטריסטיקות

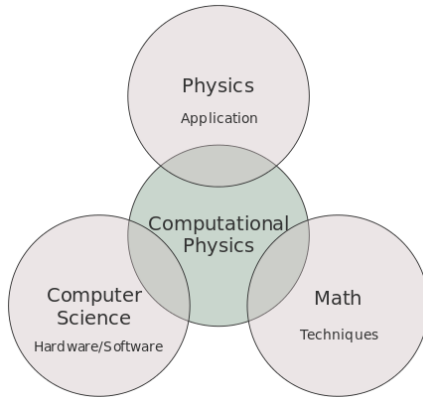
- שיטות מונטה-קרלו – אינטגרציה, שיטת מטרופוליס, מעבר של חלקיקים בחומר
- מבוא לחישוב מקבילי – OpenMP לזכרון משותף, MPI לזכרון מבוזר

ברוב נושאי הלימוד יינתנו דוגמאות של בעיות פיזיקאליות. ליבת הקורס הם התירגולים העצמיים של המשתתפים.

הקורס יתבסס על Python 3 ומניח ידע בכתיבת תוכניות בשפה זו. כמו כן שנשתמש בחבילות המדעיות numpy ו-scipy ובחבילת הגרפיקה matplotlib, השימוש בחבילות אלו הינו פשוט במיוחד למי שמכיר matlab.

חובת ההגשה של התרגילים היא 100%. תדירות התרגילים היא אחד לשבוע בתחילת הקורס ואחד לשבועיים בהמשכו. תרגיל מעט יותר מקיף יינתן בסוף הקורס. התרגילים הם עצמיים (אין לעשותם בקבוצות!). משקל התרגילים בציון הסופי נע בין 7.5% ל-25%. כשמגשים תרגיל יש להגיש גם את קוד הפיתוח וגם את קבצי הפלט והגרפים המבוקשים (בחלק מהתרגילים) וכן הסבר מילולי על התוצאות. דרך מועדפת היא להשתמש ב-jupyter ולהגיש את קובץ ה"המחברת" (*ipynb), דרך חלופית היא להגיש קבצי פיתוח ובנפרד קובץ המאגד את קובץ הפלט, גרפים והסברים כקובץ pdf.

מבוא לפיזיקה חישובית



פיזיקה חישובית היא תחום שהתפתח בצורה משמעותית עם הגידול העצום במידת הפופולריות של מחשבים אלקטרוניים ובגידול המרשים בכוחם. אולם תחום זה נסמך בחלקו על שיטות של אנליזה נומרית שהיו קיימות כבר לפני שהיו מחשבים. חלק מהשיטות אותן נלמד פותחו לפני מאות שנים, על ידי אנשי מדע ידועים כגון ניוטון גאוס ויעקובי.

כשמתייחסים לפיזיקה חישובית מתכוונים לשני סוגים של חישובים. הראשון הוא שימוש באלגוריתם נומרי לצורך פתרון של מודל פיזיקאלי כגון חישוב של גודל פיזיקאלי מעניין על ידי ביצוע אינטגרל נומרי, או על ידי פיתוח לטור כאשר כל סדר בפיתוח מכיל מספר איברים הולך וגדל. את הסוג השני של החישובים ניתן להגדיר כסימולציה בה מגדירים חוקים פיזיקאליים פשוטים יחסית ומשתמשים בנומריקה כדי לבחון כיצד חוקים אלו באים לידי ביטוי במערכת מורכבת.

פיזיקה חישובית יכולה להיחשב כתחום נפרד בנוסף לתחומים המסורתיים של פיזיקה תיאורטית ופיזיקה ניסויית. המטרה של חקר פיזיקאלי היא לפתח הבנה של התהליכים הרלבנטיים. בפיזיקה תאורטית הדבר נעשה על ידי מודלים הנפתרים אנליטית ובפיזיקה ניסויית על ידי התבוננות מבוקרת בתהליכים עצמם. פיזיקה חישובית דומה לפיזיקה תיאורטית בכך שהיא מתייחסת למודלים של התהליכים ודומה לפיזיקה ניסויית בכך שמתבוננים בצורה מבוקרת בהתנהגות המודלים.

כאמור לעיל, חלה התפתחות גדולה בתחום זה בשבעים השנים האחרונות: פותחו שיטות נומריות חדשות (כגון מונטה-קרלו ודינמיקה מולקולרית), התגלו תופעות פיזיקאליות חדשות וההבנה לגבי תופעות רבות אחרות גדלה.

בפתרון בעיות על-ידי מחשב יש להתחשב בשלושה גורמים: שגיאות (עיגול), דיוק וציביות.

שגיאות עיגול –

נובעות מצורת ההצגה של מספרים במחשב (נקרא גם roundoff error). הצגת המספרים עשויה להיות שונה בין שפות תכנות שונות ובין יישומים שונים של אותה שפה. שני סוגים עיקריים הם: שלמים integer ממשיים real. במקרים רבים שלמים מיוצגים על ידי `integer(4)` כלומר, מיוצג על-ידי ארבעה בתים שהם 32 סיביות בינריות, מהן אחת לסימן $\leq$ טווח אפשרי הוא $\pm 2 \cdot 10^9$ (ערכים מקורבים).

ההצגה של מספר שלם היא מדויקת וכך גם התוצאה של פעולה אריתמטית בין מספרים שונים, בתנאי שהתוצאה אינה חורגת מהטווח המותר. ובתנאי, שפעולת חילוק משמעה, הערך השלם של התוצאה. בפייתון 3 אין מגבלה על הערך המרבי של שלמים (אם כי ב-numpy הצגת השלמים נסמכת על הצגת long של c, כלומר integer המיוצג על ידי 64 סיביות או 8 בתים, ולכן אין לחרוג מהערכים המרביים והמזעריים המותרים).

ישנן שתי הצגות עיקריות לממשיים עם ארבעה בתים `real(4)` (שקול ל-32 סיביות: `float32`) ועם שמונה בתים `real(8)` (`float64`). בהצגה עם ארבעה בתים משתמשים בדרך כלל ב-24 סיביות לשמירת המנטיסה, ובשמונה סיביות לאקספוננט, כולל סיביות הסימן. מכאן, שהטווח האפשרי במקרה זה הוא בערך $\pm 10^{\pm 38}$. ברור שההצגה של מספרים ממשיים אינה מדויקת והדיוק היחסי הוא בערך 10^{-7} . כלומר, עבור הצגה זו של יש כ-7 ספרות דיוק (בהצגה עשרונית).

בהצגה של שמונה בתים יש 53 סיביות למנטיסה, ו-11 סיביות לאקספוננט. ומכאן, תחום של $\pm 10^{\pm 308}$ עם דיוק של 10^{-16} . בחישובים נומריים משתמשים בדרך כלל בהצגה של שמונה בתים עבור מספרים ממשיים. את המגבלות השונות של הצגות המספרים ניתן לקבל בפייתון על ידי הפעולות הבאות:

```
In [1]: import sys
print(sys.float_info)

sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-308, min_exp=-1021, min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2, rounds=1)
```

```
In [2]: ▶ import numpy as np
print(np.finfo(np.float))
```

Machine parameters for float64

```
-----
precision = 15    resolution = 1.000000000000001e-15
machep = -52     eps = 2.2204460492503131e-16
negep = -53      epsneg = 1.1102230246251565e-16
minexp = -1022   tiny = 2.2250738585072014e-308
maxexp = 1024    max = 1.7976931348623157e+308
nexp = 11       min = -max
-----
```

```
In [4]: ▶ print(np.iinfo(np.int))
```

Machine parameters for int64

```
-----
min = -9223372036854775808
max = 9223372036854775807
-----
```

```
In [5]: ▶ import numpy as np
print(np.finfo(np.float32))
```

Machine parameters for float32

```
-----
precision = 6    resolution = 1.0000000e-06
machep = -23     eps = 1.1920929e-07
negep = -24      epsneg = 5.9604645e-08
minexp = -126   tiny = 1.1754944e-38
maxexp = 128    max = 3.4028235e+38
nexp = 8        min = -max
-----
```

```
In [6]: ▶ print(np.iinfo(np.int32))
```

Machine parameters for int32

```
-----
min = -2147483648
max = 2147483647
-----
```